



Bachelorarbeit: Experimentelle Analyse von Algorithmen für fair-k-center mit Ausreißern

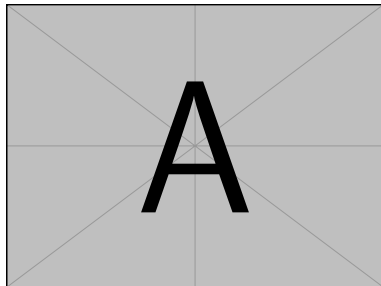
Carsten Krollmann

1 Einleitung

2 Algorithmen

- zur Analyse von Daten
 - unsupervised
 - Nähe von Datenpunkten finden
- Zielfunktion hier k-center
 - maximaler Radius minimieren
 - Zentren aus der Punktemenge
- optimales k-center $\in NP^a$
- Algorithmus von Gonzalez als bekannter Vertreter

^a[Melanie Schmidt](#). „Kombinatorische Algorithmen für Clusteringprobleme - Vorlesungsskript“. 2023.

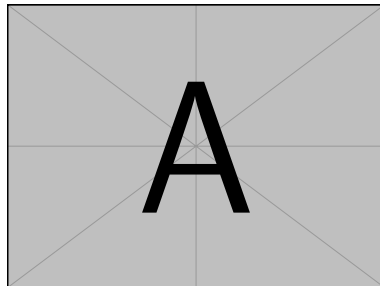


- Approximationsgarantie von 2
- colorblind

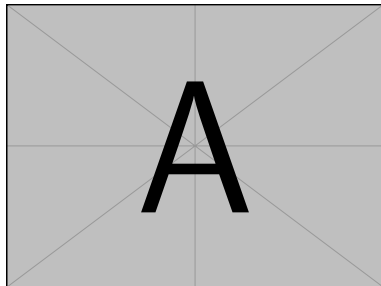
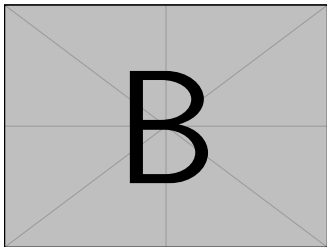
```

1:  $Z :=$  leere Zentrenmenge
2: Wähle erstes Zentrum  $z_0$  zufällig
3: for all  $x \in P$  do
4:    $d_{min} = dist(z_0, x)$ 
5: end for
6: for  $i \in 0, \dots, k - 1$  do
7:    $z_i = P(argmax(d_{min}))$ 
8:   for  $x \in P \setminus Z$  do
9:     Prüfe ob Punkte näher an neuem Zentrum
10:   end for
11: end for
12:  $r_{max} =$  größter Abstand zwischen Punkt und seinem Zentrum
13: return  $(Z, P, r_{max})$ 

```



- faires Attribut
- Zusatzbedingung: gleichmäßig aufteilen auf Cluster
- optimales fair k-center ebenfalls $\in NP^a$



^a[Melanie Schmidt](#). „Kombinatorische Algorithmen für Clusteringprobleme - Vorlesungsskript“. 2023.

- vergleich 2 Verschiedener Ansätze
 - Red-Clustering^a
 - Fast-Anchor-Clustering^b
- Nutzen von Fairlets
 - kleinst mögliche Clustereinheit
- beschränkt auf 2 Farben
- beide Algorithmen verallgemeinbar
- Aufteilung in 1:1 Cluster

^aFlavio Chierichetti u. a. *Fair Clustering Through Fairlets*. 2018. arXiv: 1802.05733 [cs.LG].

^bIrina Fast. „Fair k-Center Clustering mit Ausreißern“. Düsseldorf, NRW, Germany, Dez. 2023.

- vergleich 2 Verschiedener Ansätze
 - Red-Clustering^a
 - Fast-Anchor-Clustering^b
- Nutzen von Fairlets
 - kleinst mögliche Clustereinheit
- beschränkt auf 2 Farben
- beide Algorithmen verallgemeinbar
- Aufteilung in 1:1 Cluster

Zusatz: Ausreißer

- reelle Daten nicht perfekt 1:1
- so viele Ausreißer sodass 1:1
 - werden nicht mehr in Cluster berücksichtigt
- bei $n : n + k$ Verteilung $\rightarrow k$ Ausreißer

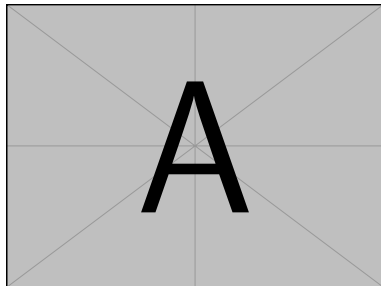
^aFlavio Chierichetti u. a. *Fair Clustering Through Fairlets*. 2018. arXiv: 1802.05733 [cs.LG].

^bIrina Fast. „Fair k-Center Clustering mit Ausreißern“. Düsseldorf, NRW, Germany, Dez. 2023.

- kleinste, faire Clustereinheit
 - hier 1:1 (Rot : Blau)
- Nutzen von Fairlets
 - Fairlets immer zusammen zu Clustern hinzufügen
 - → Cluster immer ausgeglichen
- können optimal berechnet werden über Matching
- Unterschied zwischen Red-Clustering und Fast-Anchor-Clustering

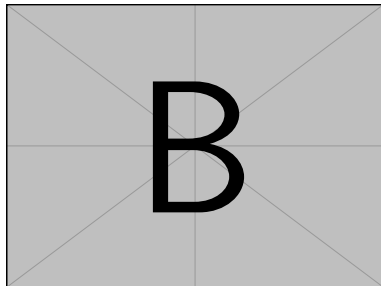
Finden der Fairlets

- bipartiter Graph mit Roten und Blauen Knoten aufbauen
- mit Quelle s und Senke t zum Flussnetzwerk machen
- Kanten $s \rightarrow \text{Rot}$ und $\text{Blau} \rightarrow t$ einfügen
- Kanten zwischen den Mengen nach Bedingung:
 - $r_{\text{test}} \geq \text{dist}(a, b)$ dann Kante
- maximaler Fluss berechnen
- bestes Überdeckendes Matching suchen
- gematchte Punkte sind Fairlets
- nicht gematchte Punkte $\rightarrow$ Ausreißer



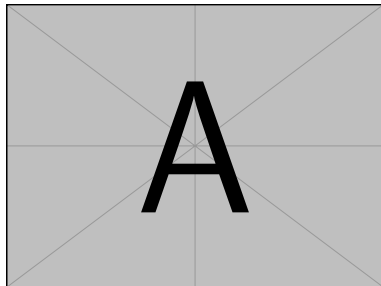
Clustern mit Fairlets

- nur Rote Punkte Clustern
- alle Blauen Punkte zu ihren Partnern zuweisen



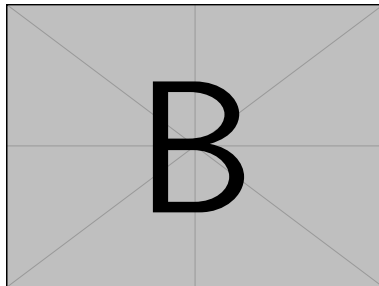
Finden der Fairlets - Anchor Variante

- Anker für jedes Knotenpaar bestimmen
 - $\rightarrow$ max Abstand der Partner zum Anker minimal
- bipartiter Graph mit Roten und Blauen Knoten aufbauen
- mit Quelle s und Senke t zum Flussnetzwerk machen
- Kanten $s \rightarrow$ Rot und Blau $\rightarrow t$ einfügen
- Kanten zwischen den Mengen nach Bedingung:
 - $r_{test} \geq anchor - dist(a, b)$ dann Kannte
- maximaler Fluss berechnen
- bestes Überdeckendes Matching suchen
- gematchte Punkte sind Fairlets
- nicht gematchte Punkte $\rightarrow$ Ausreißer



Clustern mit Fairlet-Anker

- alle Punkte ohne Ausreißer Clustern
- Zentren und ihre Fairletpartner sich selbst zuweisen
 - → Center-Awareness
- Punkte dem nächsten Zentrum ihres Ankers zuweisen



Red-Clustering vs Fast-Anchor-Clustering (Theorie)

- Red-Clustering hat Approximationsgarantie von 4
- Fast-Anchor-Clustering hat Approximationsgarantie von 3
- Laufzeit beider Algorithmen $\in \mathcal{O}(n^3)$